Содержание

Power Systems Agent Benchmark: перенос исполняемой оценки на ИИ-агентов в задачах электроэнергетики **1**
Аннотация . . . 1
1. Введение . . . 2
2. Связанные работы . . . 3
3. Требования к бенчмарку для электроэнергетических агентов . . . 4
4. Архитектура . . . 4
4.1 Контракт артефактов . . . 4
4.2 Агент-агностичный запуск и устойчивость harness . . . 5
4.3 Лестница зрелости оценщиков . . . 5
4.4 Скрытый срез как генератор, а не как файлы . . . 5
5. Набор задач . . . 6
5.1 Доменная выверка набора . . . 6
6. Проверяющие программы и метрики . . . 6
7. Воспроизводимость . . . 7
8. Эксперименты . . . 7
8.1 Канонический раунд (public vs hidden) . . . 7
8.2 Типология провалов: способности против спецификации . . . 8
8.3 Целостность оценки: агент, добравшийся до оракула . . . 9
9. Ограничения . . . 9
10. Заключение . . . 9
Приложение A. Каталог задач . . . 9
Приложение B. Артефакты экспериментов . . . 10

Power Systems Agent Benchmark: перенос исполняемой оценки на ИИ-агентов в задачах электроэнергетики

Черновик препринта v2 (2026-06). Русская версия. Дословная англоязычная пара: `paper_en_v2.md`*. Аннотированная библиография:* `bibliography.md`*; BibTeX:* `references.bib`*.*

Аннотация

Исполняемая оценка — когда агент совершает действие, а программа затем проверяет последствия этого действия, а не оценивает его изложение — к настоящему моменту стала устоявшейся практикой для инструментальных ИИ-агентов: от исправления кода (SWE-bench) до взаимодействия с инструментами и API (τ-bench) и работы в терминале (terminal-bench). Электроэнергетика до сих пор оставалась в основном вне этой практики: агентов в этой области по-прежнему оценивают текстовыми вопросами или через LLM-as-a-judge, а немногочисленные доменные бенчмарки либо оценивают свободный текст (ElecBench), либо покрывают один тип задач (PFBench — только power flow). Мы переносим исполняемую парадигму на широкую оценку электроэнергетических агентов. В Power Systems Agent Benchmark агент получает структурированную задачу, возвращает структурированное решение, а детерминированный оценщик пересчитывает инженерные величины, проверяет физические и эксплуатационные ограничения и сообщает флаг допустимости, нормированный балл и явный список нарушений. Публичный срез охватывает 41 семейство задач — расчёт режимов и анализ сети, короткие замыкания и защиту, устойчивость и соответствие grid code, DER/EV и накопители, диспетчеризацию микросетей, надёжность и восстановление, качество электроэнергии и стандарты, кибербезопасность измерений, прогнозирование — каждое заземлено на цитируемый источник и стандарт. Вместо того чтобы поставлять статический скрытый

срез, который стареет и утекает, мы публикуем детерминированный генератор, порождающий скрытые кейсы из приватного seed с автоматической проверкой решаемости, так что механизм аудируем, а числа остаются приватными. Наш центральный методологический результат в том, что запуск небольшой матрицы агентов и трактовка их идентичного провала как сигнала дефекта — практичная процедура контроля качества для исполняемых бенчмарков: она вскрыла десять дефектов постановок и, что особенно показательно, латентный баг физики внутри одного из наших собственных оценщиков, который проверка самосогласованности бенчмарка увидеть не могла. На исправленных постановках публичный и скрытый срезы дают согласованный рейтинг и сопоставимые баллы, поэтому мы позиционируем скрытую оценку как проспективную защиту от контаминации, а не как разовый разрыв сложности. Сегодня оценщики — компактные детерминированные суррогаты; замена их внутренности на simulator-backed реализации (pandapower, OpenDSS) — это дорожная карта, оставляющая контракт задач неизменным.

1. Введение

То, как мы оцениваем ИИ-агентов, незаметно менялось. По мере того как агенты переходили от ответов на вопросы к действиям — чтению данных, написанию вспомогательного кода, вызову инструментов и принятию решений — наиболее информативные бенчмарки перестали оценивать то, что агент говорит, и начали проверять то, что делают его действия. SWE-bench прогоняет тестовый набор проекта против патча агента; τ-bench сравнивает конечное состояние базы данных с размеченной целью; terminal-bench изучает эффект команд в реальной оболочке. Этот исполняемый стиль оценки сегодня — устоявшаяся практика, и не зря: он измеряет исходы, а не убедительность, и устойчив к смещениям, присущим языковой модели, которой поручили судить изложение другой модели.

Электроэнергетика этот сдвиг в основном пропустила. Агенты в эту область приходят — для анализа повреждений, диспетчеризации, исследований присоединения — но бенчмарки, которыми их оценивают, по-прежнему опираются на сопоставление с эталонным ответом или на оценку через LLM-as-a-judge, а немногочисленные доменные попытки сужают охват, чтобы оценка оставалась посильной: ElecBench оценивает рассуждения о диспетчеризации преимущественно через текст и судей, а PFBench применяет программную проверку, но только к power flow. Здесь это несоответствие важнее, чем где-либо, потому что электроэнергетический ответ может быть беглым и при этом неверным так, как текст выявить не способен. Предложенная уставка защиты может звучать авторитетно и не быть селективной; диспетчеризация может быть хорошо обоснована и нарушать пределы генератора; план восстановления может быть связным и обесточить не тот фидер. Ошибка проявляется не как формулировка, а как перегруженная линия, недопустимое напряжение или неверно локализованное повреждение — и распознать её способна только проверка, пересчитывающая физику.

Эта работа переносит исполняемую оценку на электроэнергетических агентов. Мы не претендуем ни на саму идею исполняемой оценки, ни на понятие инструментального агента; и то и другое унаследовано от перечисленных бенчмарков. Наш вклад — адаптация этой идеи к области, которая её не имела, причём в широте, вместе с инфраструктурой, которой такая адаптация требует, и методологией, удерживающей её честной. Конкретно: каждая задача публикует структурированные входные данные; агент возвращает машиночитаемое решение; а независимый оценщик пересчитывает целевые величины, проверяет жёсткие ограничения и возвращает флаг допустимости, нормированный балл и явный список нарушений. Свободное объяснение агента сохраняется, но намеренно выводится из контура оценки — это диагностика, а не основание для балла.

Из этого устройства следуют пять вкладов. Первый — мы инстанцируем исполняемый контракт для электроэнергетики как единый интерфейс task/submission/evaluation с JSON-схемами для каждого артефакта и агент-агностичным протоколом запуска; вклад — не идея контракта, а её конкретная, охватывающая домен реализация. Второй — мы собираем сбалансированный публичный набор из 41 исполняемого семейства по восьми доменным группам, каждое с прослеживаемым происхождением и двуязычным пред-

ставлением, так что инспектировать читатель может именно охват и заземлённость бенчмарка, а не единственный показательный пример. Третий — мы заменяем обычный статический скрытый срез детерминированным per-family генератором: скрытые кейсы порождаются из приватного seed на хосте оценки, и розыгрыш сохраняется, только если baseline семейства его решает и оценщик подтверждает допустимость, что делает склонный к утечкам скрытый набор аудируемым по механизму, но приватным по числам. Четвёртый, и самый важный, — мы показываем, что небольшая матрица агентов работает и как инструмент контроля качества: когда все агенты проваливают задачу одинаково, дефект обычно в постановке, а не в агентах, и действие по этому критерию позволило нам найти и исправить десять дефектов постановок. Пятый — сильнейший случай этого критерия обратился внутрь: он вскрыл латентный баг физики внутри одного из наших собственных оценщиков (off-by-one в модели индикатора тока КЗ), который проверка самосогласованности бенчмарка «baseline = evaluator» структурно не могла обнаружить, потому что baseline унаследовал ту же ошибку; вскрыли его три независимых агента, применивших корректную физику.

В основе всего этого лежит инфраструктура, которой, как выяснилось, честное сравнение агентов требует, и которую мы считаем самостоятельным переиспользуемым вкладом: harness, изолирующий каждый прогон так, чтобы агент не мог добраться до оракула, терпящий мусорные сабмишены без обрыва, переживающий краши оценщиков и отделяющий настоящие провалы агента от нестабильности провайдера узкой политикой ретраев, которая сообщает raw и retry-normalized метрики бок о бок (раздел 4).

2. Связанные работы

Наша работа стоит на стыке трёх направлений исследований, и наши долги перед каждым проще всего обозначить через то, где мы им следуем и где расходимся.

Первое направление — исполняемая оценка агентов. SWE-bench [jimenez2024swebench] и более ранняя конструкция pass@k в Codex/HumanEval [chen2021codex] установили, что выход агента можно оценить, запустив его; τ-bench [yao2024taubench] обобщил идею на взаимодействие с инструментами и пользователем, сравнивая конечные состояния, и ввёл pass^k как меру устойчивости; terminal-bench [terminalbench2025] и MLE-bench [chan2024mlebench] распространяют её на работу в оболочке и ML-инженерию, а общие наборы вроде AgentBench [liu2024agentbench], GAIA [mialon2023gaia] и WebArena [zhou2024webarena] зондируют широкое использование инструментов. Мы принимаем эту парадигму целиком; наша единственная претензия к ней — охват, поскольку ни один из этих наборов не нацелен на электроэнергетику и её физические ограничения.

Второе направление объясняет, почему мы держим LLM вне контура оценки. Судейство языковой моделью [zheng2023judging] удобно, но несёт документированные смещения — среди них позиционное, по длине и самопредпочтение [wang2023fairevaluators, gu2024judgesurvey, li2024generationjudgment], — и они опаснее всего именно тогда, когда неверный ответ беглый, что в инженерии — обычный случай. Поэтому мы оцениваем детерминированным пересчётом, а rationale модели понижаем до диагностики.

Третье направление — доменное. Обзоры [llmpowersurvey2025, agenticpowersurvey2025] документируют быстрый приток LLM- и агентных методов в энергетику без устоявшегося способа их оценивать. Ближайшая к нам работа — PowerAgentBench [poweragentbench], бенчмарк-компонент экосистемы PowerAgent в Harvard SEAS [zhang2025poweragent], которая так же сочетает public/hidden split со скрытым оценщиком, пересчитывающим физическую корректность, но на трёх уровнях задач системы IEEE 39-bus, а не на широком наборе семейств, и без опубликованной статьи на момент написания. ElecBench [zhou2025elecbench] оценивает диспетчерские сценарии преимущественно через текст и судей, а PFBench [pfbench2025] применяет программную проверку, но только к power flow. Появляющиеся сейчас производственные агенты — GridMind [gridmind2025], X-GridAgent [xgridagent2025], Grid-Mind CIA [gridmindcia2026], PFAgent [pfagent2026] — и есть целевые испытуемые бенчмарка вроде нашего. От этих соседей мы отличаемся охватом (41 семейство по восьми доменным группам), генерацией скрытых

кейсов из seed вместо их хранения и явным циклом контроля качества.

Ещё два корпуса работ формируют конкретные проектные решения. RL-среды управления сетью, прежде всего Grid2Op/L2RPN [marot2020l2rpn, marot2021retrospective], предлагают зрелую постановку для последовательного топологического управления; мы решаем дополнительную задачу — однократные инженерные решения по многим подзадачам под агент-агностичным контрактом — и оставляем симуляторы, на которые такие среды опираются — pandapower [thurner2018pandapower], MATPOWER [zimmerman2011matpower], OpenDSS [dugan2011opendss], PowerModels/PGLib [coffrin2018powermodels, babaeinejadsarookolaee2019pglib], ANDES [cui2021andes], PyPSA [brown2018pypsa] — как substrate, в который наши проверки позже можно подставить без изменения формата задач. Наконец, литература о целостности бенчмарков формирует наш скрытый срез: поскольку контаминация обучающих данных завышает результаты [contaminationsurvey2025, statictodynamic2025], стандартным ответом стали динамические и обновляемые бенчмарки [kiela2021dynabench, white2024livebench, jain2024livecodebench], и наш seed-генератор — prevention-механизм того же семейства: мы публикуем способ изготовления кейсов, а не кейсы. Harness, в свою очередь, следует предписаниям [kapoor2024agentsmatter] о стоимости, воспроизводимости и разделении классов провалов и уважает ограниченность best-of-N, установленную в [brown2024monkeys].

3. Требования к бенчмарку для электроэнергетических агентов

Прежде чем описывать систему, стоит зафиксировать, чему такой бенчмарк должен удовлетворять, поскольку ряд наших проектных решений — прямое следствие этих требований. Решение должно проверяться кодом, а не сравнением строк (исполнимость), и эта проверка должна пересчитывать физические величины и проверять ограничения, а не сверять с сохранённым текстом (физическая проверяемость). Публичные кейсы должны поддерживать разработку, а скрытые — честное сравнение, причём скрытые ответы исключены по построению, а не по политике (разделение public/hidden). Запуск агента не должен зависеть от конкретной модели, CLI или фреймворка — контракт сводится к «прочитай файлы, запиши файл» (агент-агностичность). Каждая задача должна объявлять, откуда она, с уровнем доверия (прослеживаемость), а каждый результат — быть машиночитаемым: `feasible`, `score $\in$ [0,1]`, `violations`, метрики (машиночитаемый результат). Бенчмарк должен уметь становиться сложнее без нарушения совместимости, так чтобы переход от формульного суррогата к simulator-backed проверке менял внутренность оценщика, но не формат task или submission (рост сложности без смены контракта). К этим семи, которые мы задали заранее, эксперименты позже добавляют восьмое, которого мы не предвидели: бенчмарк должен включать процедуру отделения дефектов спецификации от настоящих различий в способностях (контроль качества постановок, раздел 8.2).

4. Архитектура

4.1 Контракт артефактов

Бенчмарк организован вокруг единого линейного контракта,

```
task.json → агент (любой CLI) → submission.json → evaluator → evaluation-result.json
```

с JSON-схемой, фиксирующей каждый артефакт. `task.json` несёт идентичность и семейство задачи, публичные входы, схему сабмишена, жёсткие ограничения, метрики, происхождение и двуязычный блок представления; `submission.json` несёт структурированный ответ агента и опциональный rationale; а `evaluation-result.json` несёт флаг допустимости, балл, нарушения и любое подтверждающее evidence. Небольшая, но существенная деталь: некоторые семейства дублируют данные кейса в двух местах — внутри `public_inputs` и на верхнем уровне задачи — и валидатор артефактов следит, чтобы эти копии совпадали; такую проверку мы добавили после того, как расхождение между ними однажды проскользнуло.

4.2 Агент-агностичный запуск и устойчивость harness

Превратить задачу в нечто, что может попытаться решить произвольный CLI-агент, а затем доверять результату настолько, чтобы сравнивать агентов между собой, сложнее, чем кажется поначалу, и именно три свойства harness делают сравнение обоснованным. `prepare_agent_tasks.py` упаковывает каждую задачу в самодостаточный каталог — задача и короткий файл инструкций, с вырезанными примерами ответов и любыми полями-оракулами — а `run_agent_command.py` запускает одну shell-команду на пакет, передавая нужные пути через переменные окружения с таймаутом на кейс.

Первое свойство — изоляция. Каждый кейс выполняется в песочнице-копии своего пакета, размещённой вне репозитория, с этой песочницей в качестве рабочей директории и без единой переменной окружения или пути импорта, указывающих обратно на корень репозитория. Это не предосторожность, взятая абстрактно: в более раннем устройстве пакеты лежали внутри дерева репозитория, и агент добрался до оценщика и оценил себя против утёкшего оракула (раздел 8.3); перенос песочницы — это то, что закрывает этот путь. Второе свойство — устойчивость к мусорным сабмишенам. Структурно-валидный, но бессмысленный ответ — null, строки там, где нужны числа, вовсе неверная форма — возвращает `feasible=false` с пофилдовыми нарушениями вместо того, чтобы обрушить прогон; это поведение мы проверили зондом по каждому из 41 семейства, и полевой репорт ранее показал, что оно важно, когда агент, вернувший null, оборвал прогон на кейсе 27 из 41. Третье свойство — разделение классов провалов. Узкая политика ретраев отличает транзиентные инфраструктурные сбои — ошибки провайдера и транспорта, ранние тихие обрывы, сигнатуры quota и spend-limit — от провалов, которые на деле измеряют агента, и повторяет только первые; quality-провалы, нарушенные контракты и таймауты по бюджету кейса не ретраятся никогда, потому что их повтор молча превратил бы бенчмарк в best-of-N. Каждый результат хранит число попыток, исходный класс провала, выбранную попытку и сырую первую попытку, а summary прогона сообщает raw и retry-normalized метрики вместе со счётчиком неразрешённых инфраструктурных сбоев, так что нестабильного провайдера невозможно принять за способного агента.

4.3 Лестница зрелости оценщиков

Оценщикам намеренно позволено созревать на месте, по лестнице, идущей от формульного или графового суррогата через детерминированную инженерную модель к simulator-backed скрытому оценщику и в конечном счёте к стохастическому или масштабному набору сценариев. Публичные оценщики, описанные здесь, занимают первые две ступени. Лестницу делает применимой именно контракт раздела 4.1: поскольку форматы задачи и сабмишена зафиксированы независимо от того, как вычисляется проверка, внутренний шаг — скажем, замена суррогата DC-потока на pandapower AC-решение — меняет оценщик, не затрагивая ничего из того, что видит агент.

4.4 Скрытый срез как генератор, а не как файлы

Самое отличительное архитектурное решение касается скрытого среза, который мы вовсе не храним. Каталог `tasks/*/hidden/` исключён из системы контроля версий; на его месте репозиторий поставляет `generate_hidden_cases.py` — реестр детерминированных, учитывающих физику возмущений, по одному на семейство. Скрытый кейс производится из публичного `case_001` перевыборкой его числового содержания — параметров сети, погодных рядов, профилей нагрузки, места повреждения вместе с показаниями индикаторов и жалобами, которые оно влечёт — под приватным seed, и розыгрыш принимается, только если baseline семейства его по-прежнему решает и оценщик подтверждает решение допустимым, иначе выполняется детерминированная перерисовка. Оператор leaderboard держит seed в секрете, но любой обладатель репозитория может прочитать, как именно конструируются скрытые кейсы, не узнавая конкретных чисел ни одного раунда.

5. Набор задач

Публичный бенчмарк состоит из 41 семейства задач, которые мы группируем в восемь доменных областей, чтобы охват читался: анализ сети и ограничения режима (`ybus_construction`, `pf_voltage_repair`, `contingency_screening`, `corrective_transmission_switching`, `dynamic_thermal_rating`); короткие замыкания и защита (`three_phase_short_circuit`, с парой максимального/минимального тока по МЭК 60909, `earth_fault_calculation`, `breaker_relay_short_circuit`, `distance_protection_settings`, `overcurrent_protection`, `cable_ampacity_sizing`); устойчивость, grid code и инверторная генерация (`critical_clearing_time` по критерию равных площадей, `transient_stability_prediction`, `frt_compliance`, `ibr_short_circuit_frt`, `min_synchronous_share`); DER, PV, EV и накопители (`pv_volt_var`, `ev_v2g_outage_schedule`, `ev_v2g_voltage_support`, `bess_ancillary_response`, `commercial_pv_lcoe_uncertainty`); микросети и диспетчеризация (`microgrid_economic_dispatch`, `rolling_microgrid_dispatch`, `islanded_microgrid_pq_dispatch`, `dispatch_uncertainty`, `hydro_thermal_storage_uc n1_generation_commitment`); надёжность и восстановление (`flisr_restoration`, `fault_section_localization`, `fci_placement`, `fci_saidi_caidi`, `operator_breaker_load_actions`); качество электроэнергии, стандарты, активы и кибербезопасность (`en50160_voltage_compliance`, `power_quality_event_classification`, `harmonic_ieee519_compliance`, `transformer_thermal_loading` по МЭК 60076-7, `fdi_state_estimation` методом WLS с тестом наибольшей нормированной невязки, `protected_meter_placement`, `mas_frequency_handover`); и прогнозирование в условиях неопределённости (`wind_power_forecast`, `wind_prediction_interval`). Каждое семейство несёт source card и запись о происхождении с уровнем доверия, а полный каталог приведён в приложении.

5.1 Доменная выверка набора

Собрать задачи из литературы — не то же самое, что сделать их корректными, и доменная ревизия устранила четыре класса фактических дефектов, каждый зафиксирован в истории репозитория. Самыми ясными были физические: формула RoCoF поместила частоту системы в знаменатель, а не в числитель — ошибка `$f_0^2$` — так что ограничение по инерции, которое она должна была обеспечивать, никогда не было активным, и исправление восстанавливает уравнение движения и перебалансирует кейс. Другие были концептуальными: задача с меткой «обнаружение FDI» поставляла метки атак среди своих входов и сводилась к сравнению с порогом, и мы переработали её в полноценную WLS-оценку состояния с итеративным тестом наибольшей нормированной невязки — в постановке, где эффект размазывания делает наивный взвешенный тест неверным — явно выведя скрытные FDI за скоуп. Третий вид был вырожденным: критическое время отключения считывалось с готовых флагов устойчивости в таблице, и мы заменили его аналитическим расчётом по равным площадям. Последними были завышенные ссылки на стандарты: «расчёт по МЭК 60909», опускавший коэффициент напряжения с и заворачивавший сопротивление дуги в максимальный ток, занижал этот ток примерно на десять процентов в неконсервативную сторону, и мы переработали его в собственную пару max/min стандарта. Наконец, три семейства, которые при рассмотрении оказались вне электроэнергетики — два по теплоснабжению зданий и одно по дисперсионному разложению Соболя — были выведены в пользу доменов, которых набор ещё не покрывал: тепловой режим трансформатора по МЭК 60076-7, выбор сечения кабеля по трём управляющим проверкам и проверка гармоник по IEEE 519.

6. Проверяющие программы и метрики

Что возвращает оценщик, единообразно по всему бенчмарку — флаг допустимости, балл на единичном отрезке, список нарушений словами и task-specific evidence — но то, как он приходит к баллу, зависит от типа задачи. Для точного расчёта балл бинарен либо градуирован числовым допуском. Для задачи управления или оптимизации балл сочетает допустимость по ограничениям с качеством по целевой функции, а там, где оптимум не единствен — как в диспетчеризации микросетей — оценщик пересимулирует подан-

ное решение и оценивает разрыв до оптимальной стоимости, так что альтернативный оптимум равной стоимости получает полный балл, а допустимое, но более дорогое решение — частичный; это важно, потому что ранняя версия, требовавшая точного совпадения с одним зашитым оптимумом, штрафовала верные ответы — дефект, который матрица агентов позже вскрыла. Задачи классификации и локализации оцениваются против точных множеств с перечислением промахов, а задачи соответствия стандартам — как pass/fail с диагностическими полями. На уровне целого прогона мы сообщаем два агрегата, которые не обязаны совпадать — feasible count, долю задач, удовлетворяющих жёстким ограничениям, и perfect count, долю со score ровно единица — и, поскольку раздел 8 показывает, что они могут ранжировать агентов по-разному, рекомендуем сообщать оба наряду со средним баллом.

7. Воспроизводимость

Бенчмарк задуман так, чтобы его перепрогоняли, а не принимали на веру. Единая точка входа, `scripts/check.sh`, прогоняет набор юнит-тестов по оценщикам, baseline-ам, harness и генератору; валидирует каждый артефакт; оценивает все 41 публичных sample submissions по одному; и запускает публичные baseline-ы. Эти baseline-ы построены на тех же вычислительных ядрах, что и оценщики, и возвращают 41 из 41 feasible при среднем балле 0.997 — цифру, которую мы аккуратно подаём как проверку самосогласованности контракта, а не как свидетельство того, что задачи лёгкие. Различие не педантично: как показывает раздел 8.2, именно эта самосогласованность слепа к ошибкам в оценщике, потому что baseline, выведенный из неисправного оценщика, с ним согласуется по построению. Непрерывная интеграция прогоняет полную проверку на каждый коммит.

8. Эксперименты

Мы оценили три CLI-агента при идентичных промптах, таймауте 600 секунд на кейс и политике ретраев из раздела 4.2, без какой-либо адаптации агентов под бенчмарк: Codex CLI 0.139.0 на gpt-5.5 при medium reasoning effort, Cursor agent CLI на composer-2.5 и OpenCode 1.15.13 на deepseek-v4-flash.

8.1 Канонический раунд (public vs hidden)

Эталонный раунд был проведён после каждого исправления, описанного в этой работе. Каждый агент пытался решить зафиксированный публичный `case_001` каждого семейства и один свежесгенерированный скрытый `case_401` на семейство, взятый из нового приватного seed; ни один прогон не наткнулся на неразрешённый инфраструктурный сбой, и поэтому raw и retry-normalized метрики совпадают.

агент (модель)	public feasible	public perfect	public mean	hidden feasible	hidden perfect	hidden mean
codex (gpt-5.5 medium)	40/41	36/41	0.971	40/41	37/41	0.969
cursor (composer-2.5)	39/41	35/41	0.947	40/41	36/41	0.969
opencode (deepseek-v4-flash)	37/41	35/41	0.900	38/41	35/41	0.920

Заметны две вещи. Бенчмарк различает агентов: codex и cursor, оба около среднего 0.97, заметно выше opencode на 0.90–0.92, и делают это на обоих срезах. И на исправленных постановках публичный и скры-

тый срезы сопоставимы — hidden-балл каждого агента ложится в пределах шума от его public. Мы сообщаем это прямо, потому что более ранний, до-исправлений dev-раунд, казалось, показывал обратное, с обвалом лидера public на скрытых кейсах; то расхождение оказалось артефактом дефектов постановок из раздела 8.2, которые случайно легли тяжелее всего на одного агента, и оно исчезло, как только эти дефекты и баг оценщика были исправлены. Поэтому мы не заявляем разовый разрыв сложности между public и hidden. Довод в пользу скрытого среза — иной, проспективный: опубликованный набор стареет по мере того, как модели на нём обучаются, и генератор — это то, что удерживает доступным свежий, неконтаминированный розыгрыш. Как косвенное подтверждение: эти свежие модели не показывают на опубликованном public-наборе балл выше, чем на свежесгенерированных hidden-кейсах — признак, ожидаемый от запоминания, при таком размере кейсов попросту отсутствует.

8.2 Типология провалов: способности против спецификации

Читать провалы между агентами, а не по каждому в отдельности, оказалось самым полезным, что мы сделали, потому что провалы чисто разделяются на два рода. Когда агенты проваливают задачу по-разному — opencode пробивает лимит напряжения в Volt-VAR-управлении там, где другие его держат, неверный вердикт fault-ride-through, перепутанный знак разряда EV, расходящиеся ошибки в risk-скоринге устойчивости — разногласие есть сигнал, и именно его лидерборд должен вознаграждать. Но когда каждый агент проваливает задачу одинаково, более вероятное объяснение в том, что неверна задача, а не агенты. Трактовка этого паттерна как диагностики вскрыла десять дефектов постановок за два прохода: поле, чьё описание противоречило его имени, так что два агента верно вычислили долю там, где оценщик хотел процент; вложенные ключи, не совпавшие с проверкой; семантически пустую нулевую запись, которую тем не менее отвергли; уже упомянутое требование точного совпадения с одним из нескольких равных по стоимости оптимумов; невысказанные конвенции кривой мощности ветра, квантилей и членов теплового баланса провода; зашитую политику диспетчеризации; и неугадываемую формулу risk-скоринга. В каждом случае исправление подтверждалось перепрогоном затронутого семейства или переоценкой существующих сабмишенов и наблюдением того, как агенты восстанавливаются. Мы предлагаем лежащее в основе правило — идентичный провал всех участников следует заподозрить как дефект спецификации или оценщика — как рутинный шаг контроля качества для исполняемых бенчмарков.

Самый значимый случай указывал не на задачу, а на оценщик, и его стоит рассказать полностью, потому что это самое острое имеющееся у нас свидетельство ценности метода. На скрытом розыгрыше `fault_section_localization` все три агента поместили повреждение на секцию s24 — вывод, согласный и с индикаторами тока КЗ, и с паттерном жалоб на отключение; оценщик настаивал на другой секции, которая на деле противоречила собственным показаниям индикаторов кейса. Агенты были правы, а оценщик неправ. Причина — off-by-one в модели индикатора оценщика: он поместил нижний узел повреждённой секции на путь тока КЗ, так что для индикатора в этом узле предсказывалось, что он увидит ток КЗ, которого на деле он бы не увидел, тогда как индикатор регистрирует ток КЗ только когда повреждение лежит строго ниже него. Что делает это большим, чем исправленный баг, — это что собственные предохранители бенчмарка не могли его поймать. Baseline, которым бенчмарк самопроверяется, выведен из той же функции, что и оценщик, поэтому baseline и оценщик согласовались друг с другом, и публичный sample проходил; ошибка была невидима изнутри и стала видимой только потому, что три независимых агента привлекли корректную физику. Иными словами, исполняемый бенчмарк может таить латентный доменный дефект в самом коде, который определяет корректность, а матрица агентов — это способ его найти; предостережение, которое мы распространили бы на любой бенчмарк, чьи эталонные решения выведены из его оценщика.

8.3 Целостность оценки: агент, добравшийся до оракула

Отдельный урок пришёл из аудита логов прогона, а не баллов. На задаче устойчивости один агент — Cursor — нашёл и запустил оценщик и оценил себя против оракула; факт, который его собственный лог фиксирует словами «оценивается как feasible со score 1.0». Это стало возможным лишь потому, что в том более раннем устройстве пакеты лежали внутри рабочего дерева репозитория, а harness запускал агентов с репозиторием в качестве их рабочей директории, так что правило не обращаться к оценщику существовало только как предложение в инструкциях. Изоляция на уровне инструкций, как делает ясным этот эпизод, вовсе не изоляция для агента с автономным доступом к файловой системе, и именно она привела к песочнице раздела 4.2. Мы сообщаем это не как конфуз, а как самостоятельный методологический результат.

9. Ограничения

Несколько ограничений очерчивают, что эти результаты подтверждают. Публичные кейсы компактны — сети из трёх-пяти узлов, короткие временные ряды — поэтому они проверяют, владеет ли агент методом, а не масштабируется ли он на промышленные объёмы данных. Оценщики — детерминированные суррогаты, и никакое утверждение о промышленной точности AC не оправдано, пока на месте не окажутся simulator-backed проверки из дорожной карты. При одном кейсе на семейство в каждом срезе ранжирование по 41 точке статистически хрупко, хотя генератор делает наращивание выборки недорогим. 41 из 41 у baseline — это проверка самосогласованности контракта, а не нижняя граница сложности, и — как сделал наглядным раздел 8.2 — она не способна валидировать оценщик. Seed любого скрытого раунда расходуется, как только раунд раскрыт, поэтому цитируемый leaderboard требует свежего seed на изолированном хосте. Песочница изолирует переносом, а не контейнеризацией, что устраняет утечку через относительные пути и окружение, но не намеренный поиск оракула по абсолютному пути, поэтому соревновательному применению понадобились бы контейнеры на уровне хоста. Мы прогоняем одну конфигурацию на агента и потому не оцениваем дисперсию сэмплинга LLM. Наконец, попытка оценить собственную модель мейнтейнера на аккаунте с общим бюджетом оказалась инфраструктурно несостоятельной — она попросту исчерпала бюджет сессии — и была исключена; чистая самооценка потребовала бы выделенного API-ключа и явной оговорки о конфликте интересов.

10. Заключение

Мы перенесли исполняемую оценку, к настоящему моменту стандартную для программных и инструментальных агентов, в электроэнергетику, где оценка оставалась текстовой. Бенчмарк превращает электроэнергетические задачи в проверки, пересчитывающие физику, и на матрице современных агентов он различает их согласованно на публичном и скрытом срезах. Его наиболее переносимый вклад — не какая-либо отдельная задача, а метод: трактовка идентичного провала всех агентов как сигнала дефекта, которая по ходу этой работы нашла десять дефектов постановок и латентный баг физики внутри одного из наших собственных оценщиков, который бенчмарк не мог бы поймать одной лишь самосогласованностью — напоминание, что baseline, выведенный из оценщика, не способен его удостоверить. Долговременная ценность артефакта лежит в контракте, доменно заземлённом наборе задач, проверяющих программах, seed-генераторе скрытых кейсов с его проспективной устойчивостью к контаминации, изолированном harness и воспроизводимом цикле контроля качества; путь вперёд — наполнить проверки simulator-backed реализациями, не нарушая контракт, который держит всё это вместе.

Приложение A. Каталог задач

См. `docs/bilingual_task_inventory.md` (41 семейство, en/ru, с `task_id` и путями).

Приложение B. Артефакты экспериментов

- Канонический раунд: `runs/canonical_round1/` (summary и логи по агентам и срезам).
- Ранние dev-матрицы и история исправлений постановок: `runs/public_agent_smoke/REPORT.md`, `runs/hidden_agent_smoke/REPORT.md` и история коммитов репозитория.